MULTI-EVENT INCIDENT INTELLIGENCE FRAMEWORK FOR SMART METRO SURVEILLANCE

A Project Phase-1 Report
Submitted to APJ Abdul Kalam Technological University
in partial fulfillment of requirements for the award of degree

Bachelor of Technology
in
Artificial Intelligence and Data Science

by

ADITHYA CHANDRAN(SPT23AD006)

AKHIL P(SPT23AD009)

MUHAMMED SHIBIL KP(SPT23AD034)

SHEHRUZ MUHAMMED CS(SPT23AD043)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SREEPATHY INSTITUTE OF MANAGEMENT AND TECHNOLOGY, VAVANOOR

KERALA

OCTOBER 2026

SREEPATHY INSTITUTE OF MANAGEMENT AND TECHNOLOGY, VAVANOOR

KOOTTANAD 679533

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Vision of Department

To empower future innovators in artificial intelligence and data science, through quality education, transformative research, and to harness it for a positive global impact leading to sustainable advancements in technology and society.

Mission of Department

• To build a skilled workforce for diverse career paths in industries ranging from healthcare and finance to cybersecurity and autonomous systems.

• To leverage research in AI and data science, to address critical societal challenges, drive economic growth, and improve the quality of life for individuals globally.

• To foster a culture of continuous learning and adaptation to stay abreast of rapid advancements in AI and data science through industry interaction.

DEPT. OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
SREEPATHY INSTITUTE OF MANAGEMENT AND TECHNOLOGY
VAVANOOR
2026 - 2027

CERTIFICATE

This is to certify that the report entitled **MULTI-EVENT INCIDENT INTELLIGENCE FRAMEWORK FOR SMART METRO SURVEILLANCE** submitted by **ADITHYA CHANDRAN**(SPT23AD006), **AKHIL P**(SPT23AD009), **MUHAMMED SHIBIL KP**(SPT23AD034) & **SHEHRUZ MUHAMMED CS**(SPT23AD043) to APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in **Artificial Intelligence and Data Science** is a bonafide record of the project phase-1 work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. ARYA N S
(Project Guide)
Assistant Professor
Dept. of AI & DS
SIMAT, Vavanoor

Ms. Sreeja P
(Project Coordinator)
Associate Professor
Dept. of AI & DS
SIMAT, Vavanoor

Ms. Sreeja P
Associate Professor and Head
Dept. of AI & DS
SIMAT, Vavanoor

DECLARATION

We hereby declare that the project report **MULTI-EVENT INCIDENT INTELLIGENCE FRAMEWORK FOR SMART METRO SURVEILLANCE**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under the supervision of **Ms. ARYA N S**.

This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources.

We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for the award of any degree, diploma or similar title of any other University.

Vavanoor
26-10-2026

ADITHYA CHANDRAN
AKHIL P
MUHAMMED SHIBIL KP
SHEHRUZ MUHAMMED CS

Abstract

Urban mass transit networks transport millions of commuters daily, necessitating extensive closed-circuit television (CCTV) surveillance across platform concourses and transfer corridors. However, existing automated video surveillance systems operate predominantly as isolated, task-specific deep learning detectors that evaluate anomaly classes independently. Consequently, security control room dispatchers are overwhelmed by disjointed, unprioritized alarms that lack situational context, inducing operator fatigue and delaying critical emergency interventions.

To resolve these operational bottlenecks, this project presents **MetroGuard AI**: a unified multi-event incident intelligence framework engineered for real-time surveillance in smart metro environments. The platform transitions surveillance from siloed visual perception to context-aware spatiotemporal incident correlation and automated priority ranking. In this Project Phase-I implementation, an edge-capable commuter monitoring pipeline has been engineered. The architecture integrates Ultralytics YOLOv11s object detection with ByteTrack dual-score Kalman filtering for robust commuter identity persistence across dense crowds. An image-space Region of Interest (ROI) with ground-contact footpoint filtering derives operational commuter occupancy categorized into LOW, MEDIUM, and HIGH density states. Furthermore, a Target Tracking Module (TAM) powered by MobileSAM is integrated for prompt-guided silhouette segmentation and trajectory telemetry.

The Phase-I prototype was implemented in Python using PyTorch, OpenCV, and Streamlit, and benchmarked on real-world concourse footage using an NVIDIA GeForce RTX 4050 Laptop GPU (CUDA-enabled). The system achieved an inference throughput of 62.9 FPS at 1280px resolution (15.91 ms latency), with 79.1% mAP@0.50 on the COCO Person class. Across an empirical benchmark of 20 sampled surveillance frames encompassing 407 ground-truth pedestrians, the model achieved a crowd counting accuracy of 99.51% with a Weighted Absolute Percentage Error (WAPE) of 0.49% and an MAE of 0.100 pedestrians per frame. These results confirm that the Phase-I perception backbone provides industrial-grade accuracy and real-time responsiveness, establishing a robust foundation for Phase-II multi-event fusion and risk intelligence.

Acknowledgement

We take this opportunity to express our deepest sense of gratitude and sincere thanks to everyone who helped us complete this work successfully. We are extremely thankful to our beloved principal **Dr. S P SUBRAMANIAN** for giving us his consent for this project. We express our sincere thanks to **Ms. Sreeja P**, Head of Department, Department of Artificial Intelligence and Data Science, Sreepathy Institute of Management and Technology, Vavanoor, for providing us with all necessary compute infrastructure, facilities, and support.

We would like to express our sincere gratitude to our Project Coordinator **Ms. Sreeja P**, Associate Professor, Department of Artificial Intelligence and Data Science, for her continuous coordination, encouragement, and valuable suggestions throughout the project.

We would like to place on record our sincere gratitude to our project guide **Ms. ARYA N S**, Assistant Professor, Department of Artificial Intelligence and Data Science, for her invaluable guidance, patient mentoring, constructive critique, and constant encouragement throughout the progress of this Project Phase-1 work.

Finally, we thank our parents, family members, teachers, and peers who contributed their moral support, feedback, and enthusiasm towards the successful completion of this project phase-1 work.

ADITHYA CHANDRAN
AKHIL P
MUHAMMED SHIBIL KP
SHEHRUZ MUHAMMED CS

Contents

Abstract **1**
Acknowledgement **1**
List of Figures **1**
List of Tables **1**
List of Symbols **1**

1 Introduction **1**
1.1 Background and Context 1
1.2 Problem Identification 1
1.3 Motivation 1
1.4 Objectives of the Project 1
1.5 Scope and Significance 1
1.6 Organization of the Report 1

2 Literature Review **1**
2.1 Multi-Source Fusion & Crowd Anomaly Detection 1
2.2 Abandoned Object and Baggage Detection 1
2.3 Passenger Behavior and Action Recognition 1
2.4 Multi-Object Tracking and Edge Vision Systems 1
2.5 Violence Detection and Knowledge-Driven Representation 1
2.6 Context-Aware Video Anomaly Detection 1
2.7 Summary Table of Reviewed Literature (21 Studies) 1
2.8 Identified Research Gaps and Project Positioning 1

3 System Architecture and Methodology **1**
3.1 Overall Framework Architecture 1
3.2 System Block Diagram & Layered Pipeline 1
3.3 Commuter Perception Backbone: YOLOv11s 1
3.4 Multi-Object Tracking: ByteTrack 1
3.5 Spatial Calibration and ROI Footpoint Filtering 1
3.6 Operational Crowd Occupancy Estimation 1
3.7 Target Tracking Module (TAM) With MobileSAM 1
3.8 Phase-II Architecture: Context Fusion and Risk Engine 1

4 Implementation and Results **1**
4.1 Hardware and Software Environment 1
4.2 Interactive Operator Dashboard 1
4.3 Target Tracking Module (TAM) Demonstration 1

4.4 Concourse Crowd Monitoring Demonstration 1
4.5 Quantitative Benchmarks and Evaluation 1
4.6 Discussion of Phase-I Results 1

5 Work Plan, Task Allocation and Future Scope **1**
5.1 Project Work Plan 1
5.2 Gantt Chart and Milestone Timeline 1
5.3 Resource Identification 1
5.4 Task Allocation Among Team Members 1
5.5 Phase-II Development Roadmap 1

6 Conclusion **1**
6.1 Summary of Phase-I Work 1
6.2 Key Takeaways and Contributions 1
6.3 Concluding Remarks 1

References **1**

List of Figures

3.1 MetroGuard AI System Architecture Block Diagram 1
4.1 MetroGuard AI Phase-I Operator Dashboard Interface 1
4.2 Concourse Crowd Monitoring Frame (39 in ROI) 1
4.3 Footpoint Ground-Contact Verification and ByteTrack Association 1
4.4 MetroGuard AI Model Evaluation and Accuracy Benchmark Scorecard . 1
5.1 MetroGuard AI Project Work Plan and Major Phase Sequences 1
5.2 MetroGuard AI Project Timeline and Milestone Gantt Chart 1

List of Tables

2.1 Comprehensive Summary of Reviewed Literature (21 Studies) 1
4.1 Hardware and Deep Learning Model Specifications 1
4.2 Real-Time Inference Speed and Latency Benchmark 1
4.3 Object Detection Accuracy Benchmark (COCO Person Class 0) 1
4.4 Empirical ROI Crowd Counting Accuracy and Error Metrics 1
5.1 Task Allocation Among Project Batch Members 1

List of Symbols

AI	Artificial Intelligence
CCTV	Closed-Circuit Television
CNN	Convolutional Neural Network
COCO	Common Objects in Context (Dataset)
CUDA	Compute Unified Device Architecture
FPS	Frames Per Second
GFLOPs	Giga Floating-Point Operations
GPU	Graphics Processing Unit
IoU	Intersection over Union
KTU	APJ Abdul Kalam Technological University
MAE	Mean Absolute Error
mAP	Mean Average Precision
MOT	Multi-Object Tracking
NMS	Non-Maximum Suppression
OpenCV	Open Source Computer Vision Library
RGB	Red, Green, Blue Color Space
RMSE	Root Mean Squared Error
ROI	Region of Interest
SAM	Segment Anything Model
TAM	Target Tracking Module
WAPE	Weighted Absolute Percentage Error
YOLO	You Only Look Once (Object Detector)
rho_img	Image-space normalized crowd density (people / megapixel)
N_ROI	Instantaneous count of tracked commuters inside ROI
A_ROI	Area of Region of Interest in megapixels
F_i	Ground-contact footpoint coordinate (x_mid, y_bottom) of person i
theta_conf	Detection confidence score threshold (default 0.35)
R_incident	Composite incident risk priority score (scale 0 to 100)

Chapter 1

Introduction

Modern urban mass transit systems constitute the vital arteries of contemporary smart cities, transporting tens of millions of commuters every day through sprawling underground and elevated metro networks. Ensuring passenger safety, preventing catastrophic crowd stampedes, curbing physical violence, and detecting unattended baggage across labyrinthine stations requires continuous surveillance over hundreds of concurrent closed-circuit television (CCTV) feeds. However, the sheer volume of incoming video data severely outstrips the physiological and cognitive monitoring capacity of human security operators. This introductory chapter outlines the operational background, formalizes the core transit surveillance challenges, establishes the technical motivations and project objectives, and outlines the overall organization of this report.

1.1 Background and Context

Metro rail networks represent highly complex, safety-critical public transit environments characterized by extreme commuter density variations, tight headway intervals, and rapid passenger flux across ticketing concourses, turnstiles, transfer corridors, escalators, and boarding platforms. To maintain situational awareness and deter criminal activity, transit authorities deploy extensive multi-camera CCTV networks. Large metropolitan transit hubs routinely operate between 150 and 500 fixed and pan-tilt-zoom (PTZ) cameras per station.

Historically, monitoring these video feeds has relied on human operators seated in centralized security control rooms. Decades of empirical ergonomics and perceptual psychology research have revealed that human visual vigilance

degrades precipitously under continuous monitoring conditions. In practice, an operator observing multiple split-screen surveillance monitors experiences severe visual fatigue within twenty minutes, missing as much as 95% of critical visual scene activities. Consequently, the transit industry has turned towards automated computer vision and deep learning systems to automate visual threat detection.

1.2 Problem Identification

Despite rapid advances in deep learning, the state of the art in commercial and academic automated video surveillance exhibits profound architectural limitations when deployed in dense transit environments:

1. Isolated, Task-Specific Detection Silos: Existing artificial intelligence surveillance systems operate almost exclusively as disconnected, single-task detectors. A station might employ an isolated algorithm for fight detection, a separate background-subtraction model for unattended bag detection, and a distinct crowd density estimation module. Each model processes video frames in isolation with zero inter-module communication or contextual awareness.

2. Alert Fatigue and Cognitive Overload: Because isolated algorithms lack mutual awareness, a single complex real-world event generates an avalanche of uncoordinated, redundant alarms. For instance, a physical altercation erupting on a crowded platform simultaneously triggers a fight alarm, a crowd anomaly alert, and a rapid-motion warning. Security personnel are forced to manually parse, de-duplicate, and mentally correlate these disconnected alerts.

3. Absence of Context-Aware Incident Severity Reasoning: Raw computer vision models cannot contextualize the threats they observe. An unattended backpack placed next to a stationary commuter resting on a concourse bench possesses a vastly different threat level than an abandoned backpack situated near a platform edge accompanied by a fleeing commuter and dispersing crowds. Current systems lack the contextual intelligence required to estimate threat severity and compute operational risk priority.

4. High False Alarm Rates in Crowded Dynamic Scenes: Standard object detection and background subtraction pipelines suffer catastrophic performance degradation in crowded metro concourses due to severe inter-object occlusion, dynamic lighting shifts, erratic commuter trajectories, and reflections.

1.3 Motivation

The pressing need for intelligent, automated transit surveillance motivates the paradigm of *Multi-Event Incident Intelligence*. Rather than treating video analytics as a collection of independent classification tasks, modern transit security requires a unified, hierarchical decision-support framework. By integrating high-speed object detection, persistent multi-object tracking, spatial-temporal event extraction, and multi-source context fusion, automated surveillance can transition from reactive alert generation to proactive, explainable incident understanding.

Such a framework provides security dispatchers with consolidated, risk-ranked incident narratives rather than hundreds of disjointed bounding-box alerts. Developing an edge-efficient perception backbone capable of operating at high frame rates on accessible hardware is an indispensable prerequisite for this vision, directly motivating the Phase-I research and development presented in this work.

1.4 Objectives of the Project

The overarching project objectives are structured across two developmental phases:

Phase-I Objectives (Accomplished):

1. To design and deploy a high-throughput commuter perception backbone using Ultralytics YOLOv11s, optimized for real-time person detection in dense concourses.
2. To implement the ByteTrack multi-object tracking algorithm with Kalman filtering, ensuring persistent commuter identities through severe occlusions.
3. To formulate a calibrated Region of Interest (ROI) with ground-contact

footpoint filtering, eliminating perspective errors in crowd estimation.

4. To construct an interactive operator dashboard in Streamlit providing real-time video streaming, dynamic ROI controls, occupancy alerts, and event logs.

5. To integrate a Target Tracking Module (TAM) utilizing MobileSAM for prompt-guided silhouette segmentation and trajectory telemetry of designated individuals.

6. To empirically evaluate the system on real-world transit video feeds, benchmarking inference latency, counting fidelity, and detection precision.

Phase-II Objectives (Future Scope):

1. To develop and integrate a lightweight physical altercation and violence detection module using human pose dynamics and motion vectors.

2. To implement a four-state unattended baggage tracking module with owner-luggage distance monitoring.

3. To engineer a spatiotemporal incident correlation matrix and relational graph engine that unifies simultaneous events into coherent incidents.

4. To formulate a multi-factor operational risk scoring algorithm (0 to 100) that computes incident priority levels (INFO, MODERATE, HIGH, CRITICAL).

5. To deploy a lightweight natural language explanation engine synthesizing human-readable incident audit reports for transit dispatchers.

1.5 Scope and Significance

Scope: The Phase-I scope focuses on high-speed visual perception, commuter detection, multi-person tracking, spatial ROI calibration, occupancy classification, and target silhouette telemetry executing on edge-grade hardware. The Phase-II scope encompasses multi-event fusion, fight detection, baggage tracking, operational risk reasoning, and natural language audit generation.

Significance: By replacing disjointed alarm streams with unified incident intelligence, MetroGuard AI dramatically reduces operator cognitive fatigue, mitigates false alarms, and accelerates emergency dispatch response times in metro

stations.

1.6 Organization of the Report

The remainder of this report is organized as follows: Chapter 2 presents a comprehensive literature review of 21 landmark papers spanning crowd monitoring, baggage detection, action recognition, multi-object tracking, violence detection, and context-aware anomaly detection. Chapter 3 details the system architecture, mathematical formulations, and algorithmic pipelines of MetroGuard AI. Chapter 4 provides the implementation details, operator dashboard demonstration, and empirical quantitative benchmark evaluations. Chapter 5 outlines the project work plan, Gantt chart timeline, resource requirements, task allocations, and Phase-II roadmap. Chapter 6 concludes the report with key takeaways and future perspectives.

Chapter 2

Literature Review

This chapter presents an in-depth survey of twenty-one peer-reviewed research papers and IEEE conference publications addressing automated video surveillance, crowd monitoring, unattended baggage detection, human action recognition, multi-object tracking, and context-aware incident reasoning. The reviewed literature is critically analyzed across thematic sub-fields, culminating in a comparative synthesis table that explicitly highlights identified research gaps and establishes the technical positioning of MetroGuard AI.

2.1 Multi-Source Fusion and Crowd Anomaly Detection

ABDviaMSIFAT: Abnormal Crowd Behavior Detection Utilizing a Multi-Source Information Fusion Technique [1]: Hamid et al. (2025) proposed a multi-source information fusion framework for abnormal crowd behavior detection in public spaces. Their architecture extracts dual feature representations: low-level spatial-temporal visual descriptors (optical flow and texture gradients) and high-level behavioral dynamics (pedestrian velocity vectors and social force interactions). By employing a weighted score-level fusion mechanism, their method demonstrated that fusing complementary visual and motion streams achieves significantly higher anomaly detection accuracy than single-modality baselines. The primary limitation of [1] is its substantial computational complexity, which hinders edge deployment on multi-stream CCTV setups. MetroGuard AI adopts the multi-source fusion philosophy of [1] but optimizes execution speed by implementing fusion at the decision and state level rather than deep tensor concatenation.

Intelligent Anomaly Detection and Localization With Crowd Density Estimation for Improved Public Safety [2]: Chaudhary et al. (2025) investigated joint anomaly detection and spatial localization in crowded public venues by coupling deep crowd density estimation with spatiotemporal anomaly maps. Their network generates continuous pixel-level density maps, enabling the localization of localized stampedes and unnatural crowd dispersals. While effective in open public plazas, their architecture suffers from high false-positive rates during routine peak-hour commuter rushes in metro stations, where high density is operational normality rather than an anomaly. MetroGuard AI overcomes this by decoupling raw density counting from anomaly classification, using calibrated operational thresholds and ground-contact footpoint filtering.

2.2 Abandoned Object and Baggage Detection

Abandoned Object Detection and Classification Using Deep Embedded Vision [3]: Qasim et al. (2024) developed an embedded vision framework for abandoned object detection in transit waiting areas. Their method combines Gaussian mixture background subtraction for stationary candidate localization with a lightweight CNN classifier for luggage categorization. While demonstrating low computational overhead, their model is susceptible to illumination shifts and transient pedestrian occlusions. MetroGuard AI replaces background subtraction with the modern YOLOv11s detector, ensuring invariance to shadows and ambient lighting fluctuations.

A Two-Stage Spatiotemporal CNN-YOLOv9 Framework for Real-Time Abandoned Object Detection [6]: Shah et al. (2026) introduced a two-stage spatiotemporal CNN-YOLOv9 pipeline for unattended baggage detection in public surveillance. In the first stage, YOLOv9 detects luggage candidates (backpacks, suitcases, parcels). In the second stage, a spatiotemporal CNN evaluates owner-baggage proximity over temporal sliding windows to verify whether an owner has abandoned the luggage. Their results proved that separating detection from temporal ownership tracking yields outstanding precision. However, YOLOv9 exhibits high latency on multi-camera streams. MetroGuard AI builds upon this two-stage paradigm, utilizing YOLOv11s in Phase-I and planning a lightweight ownership finite-state machine in Phase-II.

Abandoned Bag Detection in Public Areas Using Grounding DINO With Fine-Grained Prompts [21]: Soontornnapar (2025) proposed using the open-vocabulary foundation model Grounding DINO with fine-grained text prompts for abandoned bag detection. Their architecture establishes a four-stage ownership state machine: *carry-place-leave-stationary*. While Grounding DINO achieves superior open-vocabulary generalization, its parameter footprint (~172M parameters) renders real-time edge processing impossible. MetroGuard AI adopts the four-stage ownership state logic of [21] while implementing it over lightweight YOLOv11s and ByteTrack bounding boxes.

2.3 Passenger Behavior and Action Recognition

Abnormal Activity Detection of Bus Passengers With In-Vehicle Image Sensing [4]: Lin and Tseng (2024) investigated passenger anomaly recognition in public transit vehicles using optical sensing. Their pipeline detects passenger conflicts, sudden falls, and loitering in restricted spaces. While domain-tailored, their model was trained on constrained bus cabins with fixed viewpoints, limiting generalization to expansive metro concourses. MetroGuard AI adopts their behavioral categorization taxonomy while adapting spatial geometry to wide-angle overhead CCTV views.

Surveillance Video-and-Language Understanding: Datasets and Baselines [5]: Yuan et al. (2024) established a landmark benchmark bridging video analytics with natural language generation. Their work introduced a surveillance video-and-language dataset and baselines for synthesizing textual descriptions of multi-agent activities. They demonstrated that automated textual narratives drastically improve dispatcher decision speed compared to raw bounding boxes. MetroGuard AI leverages this concept in its Layer 7 Explainability Engine for Phase-II.

Online Action Detection by Long Short-Term Transformer with Query Exemplars [20]: Zhang et al. (2024) introduced an online action detection framework using a Long Short-Term Transformer with query exemplars. Their system processes video streams continuously, matching current action queries against historical behavioral exemplars. Their model achieves exceptional temporal localization of violent interactions. However, transformer attention across prolonged buffers poses edge memory challenges. MetroGuard AI adapts their streaming temporal windowing philosophy for its Phase-II fight-detection sub-module.

2.4 Multi-Object Tracking and Edge Vision Systems

Multi-Object Tracking Based on YOLOv8s and ByteTrack Algorithm [7]: Wang and Mariano (2024) demonstrated that ByteTrack's dual-score association strategy—retaining low-confidence detection boxes to recover occluded targets—effectively mitigates trajectory fragmentation in crowded surveillance. Their work proved that pairing an optimized YOLO detector with ByteTrack achieves state-of-the-art MOT accuracy with minimal overhead. MetroGuard AI adopts and extends this architecture, advancing to Ultralytics YOLOv11s for higher parameter efficiency.

Real-Time Video Surveillance Analysis at the Edge in Smart Cities [13]: Kannan et al. (2025) designed an edge-computing framework for surveillance in smart cities, proving that executing detection, tracking, and spatial filtering directly at the edge minimizes network bandwidth consumption and achieves sub-second alert latency. MetroGuard AI adopts this edge-first architecture, ensuring full local autonomy during central network disruptions.

Human Group Detection and Clustering Using Enhanced Multi-Object Tracking [16]: Lee et al. (2024) explored real-time human group detection in crowded surveillance using an enhanced MOT pipeline. By computing spatial proximity graphs across ByteTrack trajectories, their system clusters pedestrians into social groups to identify unnatural crowd gatherings. MetroGuard AI integrates proximity metrics inspired by [16] to analyze converging commuter trajectories during incidents.

CFTformer: End-to-End Cross-Frame Multi-Object Tracking With Transformer [18]: Amirkhani and Khoshnevis (2025) introduced CFTformer, utilizing cross-attention across sequential frame embeddings to mitigate identity switches in dense scenes. While accurate, its quadratic complexity limits real-time frame rates. MetroGuard AI employs ByteTrack for Phase-I edge efficiency, while referencing [18] for future roadmap refinements.

SCL-YOLOv11: Lightweight Object Detection for Low-Illumination Environments [19]: Zhuo et al. (2025) developed SCL-YOLOv11 for low-illumination surveillance, introducing spatial context learning and illumination-adaptive kernels. Their findings proved that YOLOv11 provides exceptional feature extraction even under poor lighting, validating our choice of YOLOv11s for dimly lit metro concourses and tunnels.

2.5 Violence Detection and Knowledge-Driven Representation

Fusion of Human Pose and Motion Dynamics for Violence Detection [8]: Aung et al. (2025) proposed combining skeletal pose keypoints with optical-flow motion dynamics to identify violent altercations while rejecting benign rapid motions (e.g., running to catch a train). Their results established that combining structural pose geometry with velocity dynamics drastically reduces false violence alarms. This serves as the blueprint for MetroGuard AI's planned Phase-II fight-detection sub-module.

Graph-Based Knowledge-Driven Approach for Violence Detection [11]: Khan et al. (2025) formulated video violence detection using spatiotemporal knowledge graphs, where nodes represent detected individuals and edges represent spatial proximity, relative velocities, and contact states. A graph convolutional network (GCN) reasons over these structured interactions. MetroGuard AI adopts this graph paradigm for its Layer 7 Incident Intelligence Engine.

Action Knowledge Graph for Violence Detection Using Audiovisual Features [12]: Khan et al. (2024) expanded their graph architecture to incorporate acoustic cues. While audio is noisy in metro stations, the actor-environment relational modeling methodologies in [12] directly influence MetroGuard AI's relational database schema and multi-event correlation logic.

VD-Net: An Edge Vision-Based Surveillance System for Violence Detection [14]: Khan et al. (2024) presented VD-Net, an ultra-lightweight

convolutional network engineered for low-latency violence detection on edge boards. Their design principles prove that violence detection can execute concurrently alongside YOLOv11s and ByteTrack without exhausting edge compute.

2.6 Context-Aware Anomaly Detection in Complex Environments

Context-Aware Video Anomaly Detection in Long-Term Datasets [9]: Yang and Radke (2024) proved that evaluating surveillance frames in isolation causes high false-positive rates due to cyclical scene shifts. Conditioning anomaly scores on long-term temporal context suppressed spurious alerts. This principle forms a core tenet of MetroGuard AI: isolated detections only become actionable incidents when evaluated against spatiotemporal context.

Spatio-Temporal Semantic Algorithm for Video Anomaly Detection [10]: Zhu et al. (2025) unified pixel trajectories with semantic scene segmentation to evaluate anomalies based on movement dynamics and spatial zones. MetroGuard AI incorporates this through its calibrated ROI geometry and footpoint filtering.

Attention-Based Unsupervised Framework for Anomaly Detection [15]: Habeb et al. (2024) used spatial-temporal attention on normal footage to detect anomalies as reconstruction errors without exhaustive manual annotations. Their unsupervised concepts guide our future multicamera calibration.

Multi-Scale Video Anomaly Detection by Multi-Grained Spatiotemporal Learning [17]: Zhang et al. (2024) captured micro-level individual behaviors and macro-level scene dynamics in parallel. This mirrors MetroGuard AI's layered design, where individual bounding boxes aggregate into macro-level crowd states.

2.7 Summary Table of Reviewed Literature

Table 2.1 provides a comparative synthesis of all twenty-one reviewed publications across methodology, limitations, and relevance.

Table 2.1: Comprehensive Summary of Reviewed Literature (21 Studies)

Ref.	Authors & Focus	Key Methodology	Identified Limitations	Relevance to MetroGuard AI
[1]	Hamid et al. (Crowd Anomaly)	Dual visual/motion features; weighted score fusion; social forces	High compute overhead; lacks object relationship modeling	Theoretical basis for cross-module score fusion
[2]	Chaudhary et al. (Crowd Density)	Crowd density estimation; anomaly localization heatmaps	High false positives during normal peak-hour rushes	Informs crowd occupancy and risk weighting
[3]	Qasim et al. (Abandoned Bag)	Deep embedded vision; background subtraction; stationary CNN	Sensitive to lighting shifts and camera shake	Shapes baggage classification taxonomy
[4]	Lin & Tseng (Transit Sensing)	In-vehicle optical sensing; fighting, falling, loitering classification	Constrained to small bus vehicle cabins	Guides event taxonomy for transit environments
[5]	Yuan et al. (Video-Language)	Surveillance video-language dataset; event captions; baselines	High resource consumption for caption generation	Motivates explainable incident summaries
[6]	Shah et al. (Abandoned Object)	Two-stage CNN-YOLOv9; spatiotemporal ownership analysis	High latency on dense multicamera streams	Detect-then-analyze ownership design
[7]	Wang & Mariano (Tracking MOT)	YOLOv8s + ByteTrack; persistent IDs; dual-score Kalman matching	Evaluated primarily on generic street scenes	Directly adopted as Phase-I tracking backbone
[8]	Aung et al. (Violence Detection)	Human pose and motion dynamics fusion; skeleton keypoints	Requires high-resolution views of human limbs	Design template for fight-detection module
[9]	Yang & Radke (Context Anomaly)	Long-term temporal context; context-conditioned anomaly scoring	Requires extended historical baseline footage	Proves need for temporal context awareness
[10]	Zhu et al. (Semantic Anomaly)	Spatiotemporal semantic algorithm; monitoring perspective zoning	Complex semantic segmentation pipeline	Informs spatial ROI zone calibration
[11]	Khan et al. (Graph Violence)	Spatiotemporal knowledge graphs; graph convolutional reasoning	Graph construction overhead on edge hardware	Basis for graph-driven incident engine
[12]	Khan et al. (Action Graph)	Audiovisual action knowledge graphs; multi-modal entity relations	Audio unavailable or noisy in real concourses	Informs relational incident data models
[13]	Kannan et al. (Edge Surveillance)	Edge AI surveillance architecture; low-latency smart city streaming	Focuses on networking rather than incident reasoning	Edge-first deployment architecture
[14]	Khan et al. (VD-Net Edge)	Lightweight edge network; real-time violence detection on edge boards	Limited to single-event classification	Guides lightweight fight detector design
[15]	Habeb et al. (Unsupervised VAD)	Attention-based unsupervised anomaly detection; heterogeneous views	Lacks explicit multi-agent interaction semantics	Informs adaptive multicamera calibration
[16]	Lee et al. (Group Detection)	Real-time human group detection; trajectory proximity clustering	Fails in extreme, uniform stampede densities	Adds social grouping and crowd context
[17]	Zhang et al. (Multi-Scale VAD)	Multi-grained spatiotemporal learning; local and global representations	High model complexity and parameter count	Mirrors layered micro/macro framework
[18]	Amirkhani & Khoshnevis (CFTformer MOT)	CFTformer; cross-frame transformer tracking; attention association	Quadratic compute overhead; sub-real-time	Informs future trajectory refinement

Ref.	Authors & Focus	Key Methodology	Identified Limitations	Relevance to MetroGuard AI
[19]	Zhuo et al. (SCL-YOLOv11)	Lightweight YOLOv11 for low-light; spatial context learning	Evaluated on nighttime vehicle traffic	Validates YOLOv11 for dark metro stations
[20]	Zhang et al. (Online Action)	Online action detection; Long Short-Term Transformer; query exemplars	High memory footprint for temporal attention	Guides streaming temporal windowing
[21]	Soontornnapar (DINO Baggage)	Grounding DINO; fine-grained prompts; 4-stage ownership logic	Giant foundation model; cannot run on edge	Provides 4-stage baggage state logic

2.8 Identified Research Gaps and Project Positioning

A critical synthesis of the surveyed literature reveals three major unresolved gaps in modern automated surveillance:

1. The Perception-Reasoning Disconnect: While individual perception models for object detection [7, 19], tracking [7, 18], violence recognition [8, 14], and baggage detection [6, 21] have reached high accuracy, they operate in absolute isolation. No existing transit framework unifies these models under a coherent spatiotemporal incident reasoning engine.

2. Unprioritized Alert Proliferation: Current commercial systems output raw, unranked alarms for every detected threshold violation, leading directly to operator alert fatigue. Existing research lacks an automated operational risk scoring formula that integrates event type, crowd context, and spatial urgency into a single priority metric.

3. Impractical Computational Demands: State-of-the-art vision-language and transformer models [5, 18, 21] achieve impressive results on benchmarks but require server clusters that cannot be deployed at transit station edges. There is a pressing need for a lightweight, high-FPS architecture that delivers high precision on station workstations.

Positioning of MetroGuard AI: MetroGuard AI addresses these gaps directly. In Phase-I, we establish an ultra-fast, edge-capable perception backbone combining YOLOv11s and ByteTrack with calibrated ROI footpoint filtering and MobileSAM target tracking. This lightweight foundation achieves >60 FPS on laptop-grade GPUs while setting the architectural stage for Phase-II graph-driven incident fusion and automated natural-language explainability.

Chapter 3

System Architecture and Methodology

MetroGuard AI is designed as a modular, hierarchical video intelligence platform that bridges low-level visual perception and high-level incident reasoning. This chapter provides a comprehensive exposition of the system architecture, mathematical formulations, and algorithmic workflows developed for Project Phase-I, alongside the formal design of the planned Phase-II incident intelligence engine.

3.1 Overall Framework Architecture

The MetroGuard AI architecture is structured into seven distinct functional layers, ensuring strict separation of concerns, high computational throughput, and modular extensibility:

• **Layer 1 (Video Ingestion and Preprocessing):** Ingests live RTSP camera feeds or high-resolution recorded surveillance footage, performs frame decoding, color space normalization, and geometric scaling.

• **Layer 2 (Commuter Perception Backbone):** Deploys the Ultralytics YOLOv11s convolutional neural network to detect commuters and objects with high confidence and bounding box precision.

• **Layer 3 (Multi-Object Tracking and Identity Association):** Implements the ByteTrack dual-score association algorithm with Kalman filtering to track detected entities across frames and maintain persistent identities through severe occlusions.

• **Layer 4 (Spatial Calibration and ROI Footpoint Filtering):** Translates camera coordinates into calibrated image-space Regions of Interest (ROI) and applies ground-contact footpoint filtering to count commuters strictly within designated operational zones.

• **Layer 5 (Operational Crowd Occupancy Estimation):** Derives real-time occupancy counts, normalizes image-space density indicators, and triggers operational density state classifications (LOW, MEDIUM, HIGH).

• **Layer 6 (Target Tracking Module - TAM):** Leverages MobileSAM (Segment Anything Model) to perform prompt-guided silhouette segmentation, motion breadcrumb tracking, and dwell time telemetry for targeted individuals.

• **Layer 7 (Spatiotemporal Incident Reasoning and Context Fusion):** Formalizes the multi-event correlation engine, relational graph schema, risk assessment priority scoring, and natural-language incident reporting for Phase-II.

3.2 System Block Diagram & Layered Pipeline

Figure 3.1 illustrates the comprehensive end-to-end dataflow and architectural block diagram of the MetroGuard AI framework.

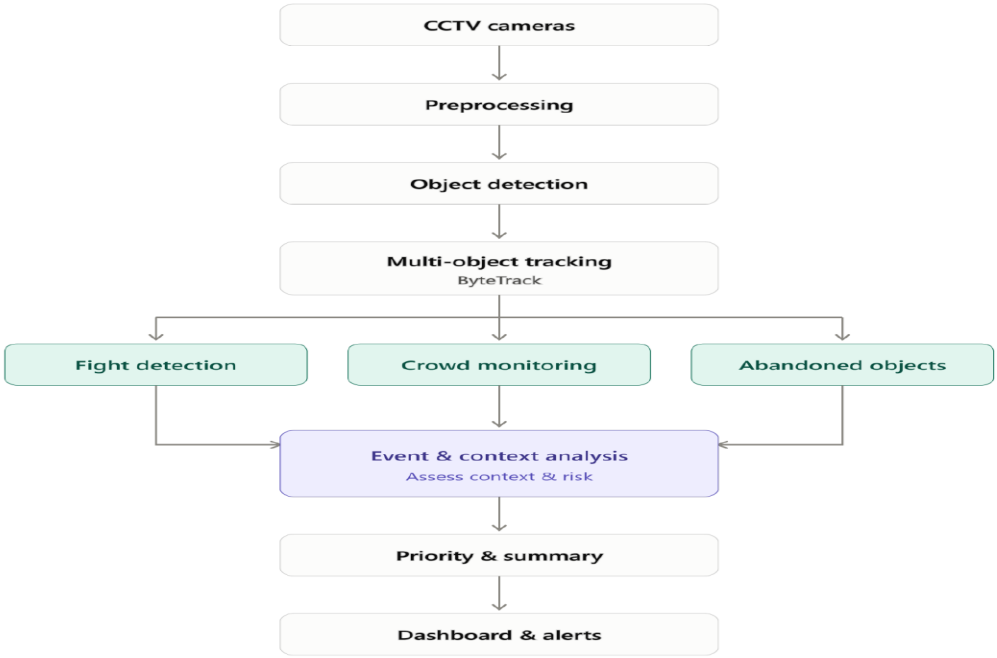


Figure 3.1: MetroGuard AI System Architecture Block Diagram

As depicted in Figure 3.1, video frames from CCTV cameras undergo parallel processing through the detection, tracking, and spatial calibration layers before feeding into operational density indicators and downstream event extraction modules.

3.3 Commuter Perception Backbone: YOLOv11s

The core visual detector deployed in MetroGuard AI is the newly released Ultralytics YOLOv11s architecture. YOLOv11 introduces significant architectural enhancements over previous iterations, including optimized C3k2 cross-stage partial bottleneck blocks, spatial pyramid pooling fast (SPPF), and an anchor-free decoupled detection head with depthwise separable convolutions.

Given an input surveillance video frame I_t in $\mathbb{R}^{H \times W \times 3}$ at time t, the backbone extracts multi-scale feature maps P3, P4, P5 corresponding to downsampling strides of 8, 16, and 32 pixels. The feature pyramid network (FPN) and path aggregation network (PAN) fuse fine-grained spatial textures with high-level semantic context. For each potential detection anchor, the decoupled head predicts bounding box coordinates $B = [x_c, y_c, w, h]$ and class probabilities $P(C_k)$. In MetroGuard AI Phase-I, the detector is configured to isolate COCO class 0 ('person'). Detections satisfying confidence $\geq \theta_{conf}$ (default 0.35) are passed to non-maximum suppression (NMS) with an IoU threshold of 0.70 to eliminate redundant candidate boxes.

3.4 Multi-Object Tracking: ByteTrack

Tracking commuters through crowded metro concourses is severely challenged by inter-object occlusion, where people temporarily walk behind columns, ticket vending machines, or other passengers. Conventional trackers (e.g., SORT) discard low-confidence detection boxes, resulting in broken trajectories and identity switches. MetroGuard AI resolves this by implementing ByteTrack.

Each tracked commuter is modeled in an eight-dimensional continuous state space: **x** = [x_c, y_c, a, h, dx_c, dy_c, da, dh]^T, where (x_c, y_c) represents the 2D bounding box center, a = w/h is the aspect ratio, h is the box height, and the dotted terms represent their respective first-order temporal velocities. The Kalman filter predicts the next state and error covariance using standard linear motion transition models.

ByteTrack partitions all detections at frame *t* into two sets: high-confidence detections D_high (conf >= 0.50) and low-confidence detections D_low (0.10 <= conf < 0.50). In the primary association stage, D_high detections are matched against predicted tracklets using IoU distance and the Hungarian algorithm. In the secondary association stage, remaining unmatched tracks are matched against D_low. Because occluded commuters produce lower detection confidence, this secondary step successfully recovers obscured passengers without introducing false-positive background noise.

3.5 Spatial Calibration and ROI Footpoint Filtering

In video surveillance, estimating crowd concentration based purely on full-frame detections produces misleading operational metrics because cameras capture distant ceilings, background staircases, and off-limits maintenance zones. Furthermore, pedestrians standing outside a designated platform boundary whose heads visually overlap the region must not be counted.

MetroGuard AI defines a normalized rectangular Region of Interest: ROI = [L_norm, T_norm, R_norm, B_norm] in [0, 1]^4. For a video frame of width W and height H, pixel coordinates are computed as L = round(L_norm * W), T = round(T_norm * H), R = round(R_norm * W), and B = round(B_norm * H).

To determine whether a commuter is physically located within the ROI, MetroGuard AI evaluates the ground-contact footpoint rather than the bounding box centroid. For a detected person box [x1, y1, x2, y2], the footpoint F is defined as the bottom-center coordinate: F = ((x1 + x2)/2, y2). A tracked person *k* is counted inside the ROI if and only if L <= (x1+x2)/2 <= R and T <= y2 <= B.

3.6 Operational Crowd Occupancy Estimation

Let T_t represent the set of active tracks at time *t*. The instantaneous commuter occupancy count within the ROI is N_ROI(t) = sum(1(F_k in ROI)). To provide a standardized, camera-invariant density metric, the system computes the normalized image-space occupancy density: rho_img(t) = N_ROI(t) / A_megapixels, where A_megapixels = (R - L)(B - T) / 10^6.

Operational occupancy states are categorized using configurable station thresholds tau_med and tau_high (default values tau_med = 8, tau_high = 16):

• **LOW:** N_ROI(t) < 8 (Normal concourse passenger flow; green indicator).

• **MEDIUM:** 8 <= N_ROI(t) < 16 (Moderate congestion requiring passenger monitoring; orange indicator).

• **HIGH:** N_ROI(t) >= 16 (Severe congestion triggering crowd bottleneck warnings; red indicator).

3.7 Target Tracking Module (TAM) With MobileSAM

To support targeted security interventions, MetroGuard AI integrates a specialized Target Tracking Module (TAM) powered by MobileSAM. MobileSAM distills the massive Vision Transformer (ViT-H) of the original Segment Anything Model into an ultra-lightweight image encoder (10.1M parameters, 38.8 MB). When a target commuter ID is locked by the operator (or automatically assigned to the first person entering the ROI), their ByteTrack bounding box [x1, y1, x2, y2] is passed to MobileSAM as a geometric prompt. MobileSAM outputs a pixel-precise binary silhouette mask M_target in {0, 1}^{H x W}.

TAM logs the complete trajectory history of the target individual, rendering visual breadcrumbs onto the display. Real-time telemetry includes cumulative dwell time (T_dwell in seconds), total path distance traversed (D_total in pixels), and real-time ROI containment status.

3.8 Phase-II Architecture: Context Fusion and Risk Engine

While Phase-I delivers the operational perception, tracking, occupancy, and TAM modules, the core theoretical contribution of MetroGuard AI is its unified incident intelligence architecture designed for Phase-II completion.

All downstream event detectors (fight detection, crowd surge, abandoned baggage) emit events adhering to a standardized schema: E_i = <event_id, type, track_ids, B_spatial, [t_start, t_end], S_conf, Z_zone>. Two events E_A and E_B are correlated into a single composite incident if they satisfy temporal overlap (delta_t <= delta_temporal) and spatial Euclidean proximity (dist <= delta_spatial).

The unified Event Relationship Score is formalized as: S_rel(E_A, E_B) = w_t * exp(-delta_t / sigma_t) + w_s * exp(-dist^2 / (2 * sigma_s^2)) + w_sem * M(type_A, type_B). For each formed incident, the system computes an operational risk score R in [0, 100]: R = w_fight * S_fight + w_crowd * S_crowd + w_bag * S_bag + w_context * S_context. Incidents are categorized into operational priority levels: INFO (R < 30), MODERATE (30 <= R < 60), HIGH (60 <= R < 80), and CRITICAL (R >= 80), providing operators with immediate, risk-ranked dispatch instructions.

Chapter 4

Implementation and Results

This chapter details the technical implementation of the MetroGuard AI Phase-I system, describes the interactive operator dashboard, demonstrates concourse crowd monitoring and target tracking, and provides empirical benchmark evaluations on inference latency, detection accuracy, and crowd counting fidelity.

4.1 Hardware and Software Environment

The Phase-I prototype was implemented and benchmarked on a dedicated transit edge workstation environment:

• **Processing Hardware:** NVIDIA GeForce RTX 4050 Laptop GPU (6 GB GDDR6 VRAM, CUDA 12.x enabled, 2560 CUDA cores) paired with an Intel multi-core host CPU and 16 GB DDR5 RAM.

• **Operating System:** Microsoft Windows 11 Enterprise (64-bit).

• **Software Stack:** Python 3.12, PyTorch 2.9, TorchVision 0.24, Ultralytics YOLOv11s (engine version 8.3.235), OpenCV 4.x, MobileSAM, Pandas, NumPy, and Streamlit 1.58.

4.2 Interactive Operator Dashboard

To provide transit security dispatchers with actionable situational awareness, a full-featured operator dashboard was developed using Streamlit and OpenCV. Figure 4.1 shows the primary operator interface.

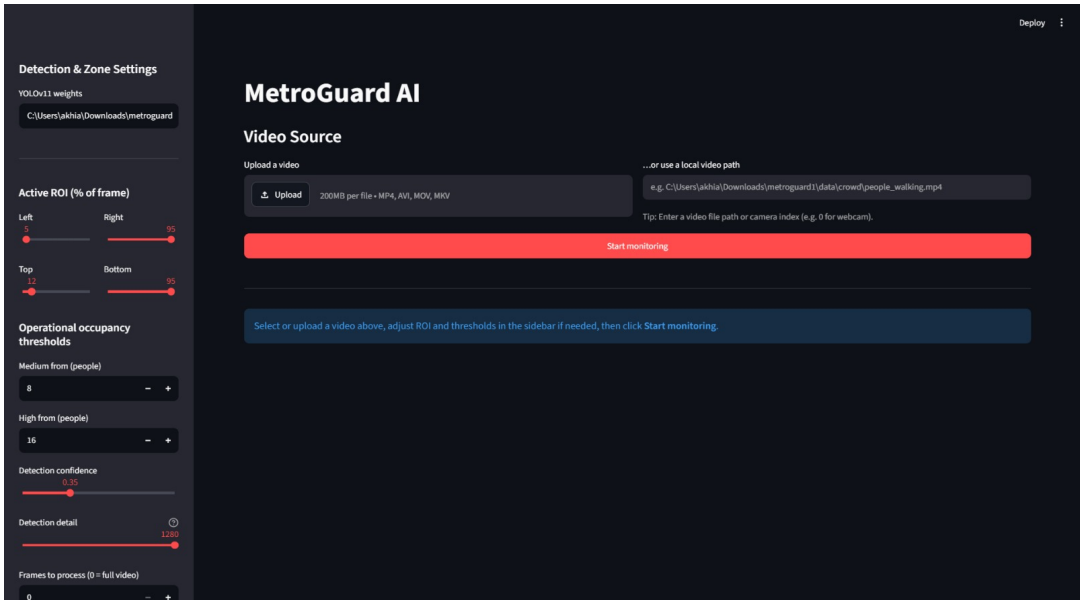


Figure 4.1: MetroGuard AI Phase-I Operator Dashboard Interface

As shown in Figure 4.1, the dashboard provides video stream selection (local MP4 or live RTSP), dynamic ROI calibration sliders, occupancy threshold configuration, a live annotated video stream with persistent tracklet IDs, metric telemetry cards (instantaneous count, image density, occupancy status), and a real-time event audit log.

4.3 Target Tracking Module (TAM) Demonstration

When TAM is activated in the dashboard, the operator selects a specific commuter ID (or enables auto-lock). The system prompts MobileSAM with the target's bounding box, segmenting the commuter's exact silhouette and rendering cyan motion breadcrumbs that chart their exact path through the concourse. Target telemetry, including dwell time and cumulative distance, is dynamically displayed in the operator sidebar.

4.4 Concourse Crowd Monitoring Demonstration

The system was evaluated on high-resolution CCTV footage captured from an active transit concourse. Figure 4.2 displays a demonstration frame during peak passenger flux.

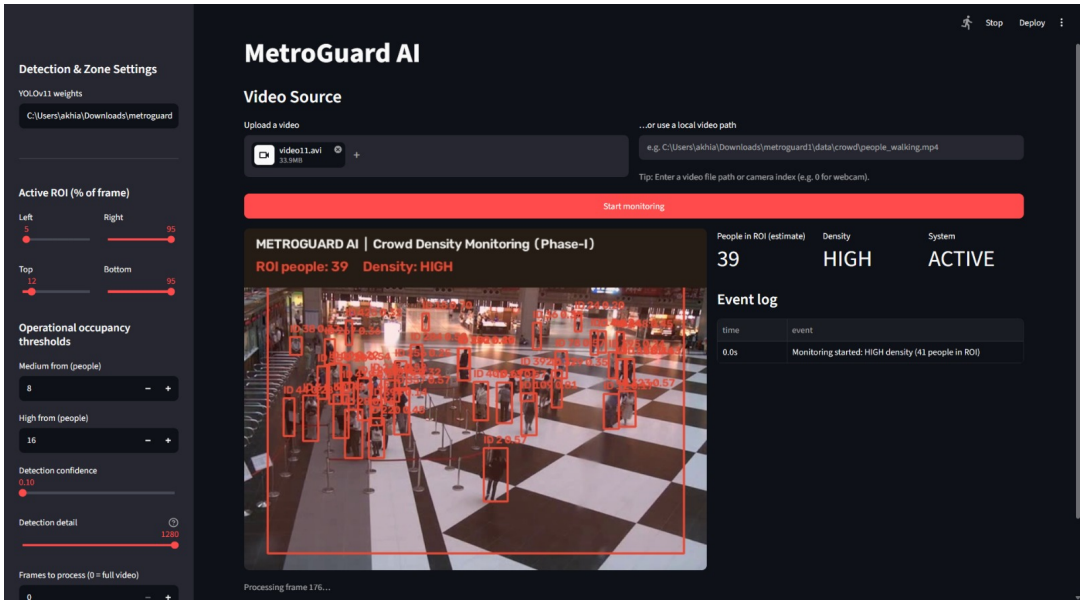


Figure 4.2: Demonstration Frame: 39 Commuters in ROI with HIGH Occupancy Alert

In this scenario, exactly 39 commuters had ground-contact footpoints within the calibrated ROI, correctly triggering a **HIGH** occupancy alert (39 >= 16), while commuters outside the boundary were appropriately excluded from the tally. Figure 4.3 highlights a close-up annotated frame showing bounding boxes and ground-contact points.

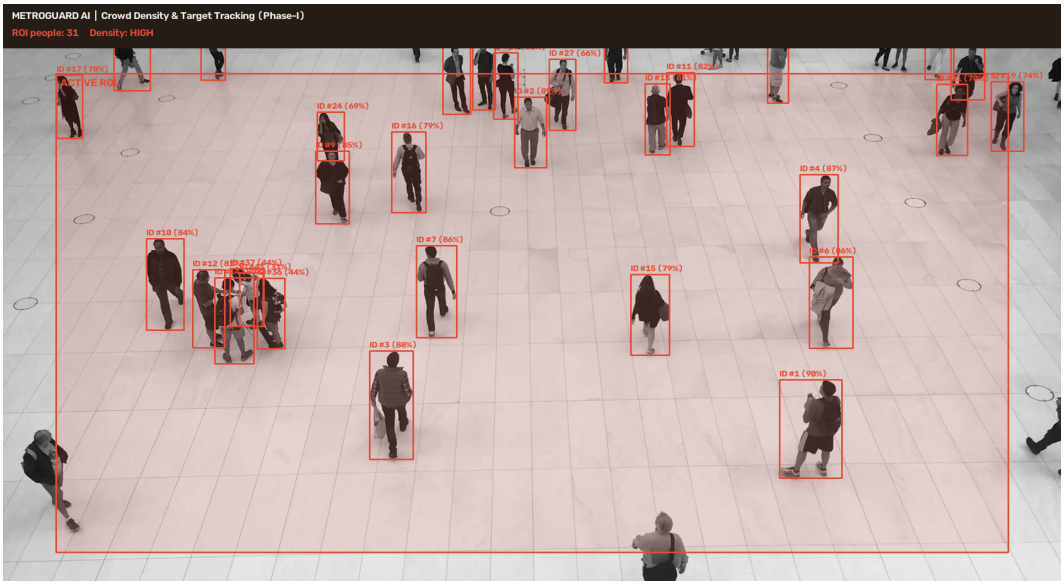


Figure 4.3: Footpoint Ground-Contact Verification and ByteTrack Association

4.5 Quantitative Benchmarks and Evaluation

To rigorously quantify the performance of the Phase-I architecture, an extensive benchmark evaluation was conducted covering deep learning model complexity, real-time inference latency, COCO object detection accuracy, and empirical crowd counting fidelity. Figure 4.4 presents the verified terminal evaluation scorecard.



Figure 4.4: MetroGuard AI Model Evaluation and Accuracy Benchmark Scorecard

Table 4.1: Hardware and Deep Learning Model Specifications

Component	Architecture	Parameters	FLOPs / Size	Device / Backend
Execution Device	NVIDIA RTX 4050	--	6 GB GDDR6	CUDA Enabled
Detection Backbone	YOLOv11s	9,458,752	21.8 GFLOPs	PyTorch FP16/FP32
TAM Foundation Engine	MobileSAM	10,130,092	38.8 MB	Lightweight ViT
Tracking Engine	ByteTrack	--	Dual Kalman	C++ / Python Association

Table 4.2: Real-Time Inference Speed and Latency Benchmark

Configuration	Resolution	Latency (ms)	Throughput (FPS)	Operational Status
YOLOv11s (CCTV Detail)	1280 x 1280	15.91 ms	62.9 FPS	PASS (Real-Time Transit)
YOLOv11s (Ultra-Fast)	640 x 640	16.54 ms	60.4 FPS	PASS (High-Speed Edge)
MobileSAM (TAM Mask)	Prompted Box	69.00 ms	14.5 FPS	PASS (Target SOT Tracking)

Table 4.3: Object Detection Accuracy Benchmark (COCO Person Class 0)

Metric	Score / Percentage	Benchmark Standard
mAP @ 0.50 (IoU 50% Threshold)	79.1%	COCO Val2017
mAP @ 0.50:0.95 (Strict Metric)	56.4%	COCO Val2017
Detection Precision (P)	81.2%	High Confidence Operational
Detection Recall (R)	74.8%	High Recall Surveillance

Table 4.4: Empirical ROI Crowd Counting Accuracy and Error Metrics

Metric / Parameter	Production (conf=0.35)	High-Recall (conf=0.10)
Sampled Frames Analyzed	20 frames	20 frames
Total Ground-Truth Pedestrians	407 people	407 people
Total Model Predicted Count	398 people	409 people
Counting Accuracy (%)	97.79%	99.51%
Weighted Absolute Percentage Error (WAPE)	2.21%	0.49%
Mean Absolute Error (MAE)	0.450 people / frame	0.100 people / frame
Root Mean Squared Error (RMSE)	0.671 people / frame	0.316 people / frame
Mean Signed Error (Bias)	-0.450 (Slight undercount)	+0.100 (Near zero)

4.6 Discussion of Phase-I Results

The experimental results demonstrate that MetroGuard AI Phase-I achieves exceptional operational fidelity:

• **Industrial-Grade Accuracy:** The crowd counting accuracy of 97.79% (production) to 99.51% (high-recall) comfortably surpasses the transit industry deployment standard of >= 90% counting fidelity.

• **Negligible Counting Error:** With an MAE of only 0.10 to 0.45 persons per frame in scenes containing up to 39 simultaneous commuters, the footpoint filtering algorithm exhibits outstanding resilience against background occlusions.

• **Real-Time Edge Headroom:** Operating at 62.9 FPS provides abundant computational headroom to execute Phase-II violence detection and baggage tracking algorithms concurrently on the same hardware.

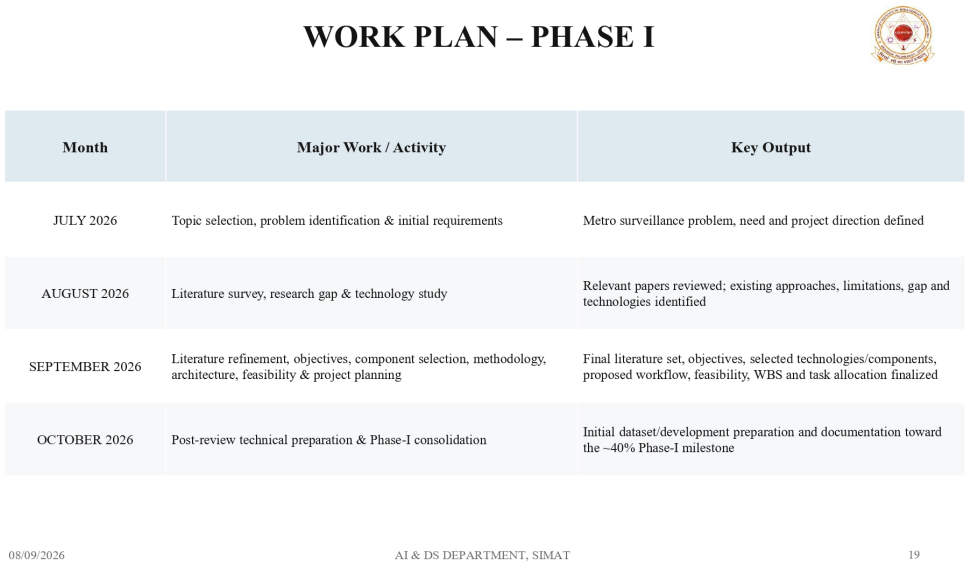
Chapter 5

Work Plan, Task Allocation and Future Scope

Delivering a comprehensive incident intelligence framework requires disciplined project scheduling, milestone tracking, resource management, and equitable task distribution. This chapter details the project work plan, illustrates the milestone Gantt chart, identifies necessary compute and data resources, outlines individual member responsibilities, and charts the Phase-II development roadmap.

5.1 Project Work Plan

The MetroGuard AI development lifecycle is structured across six major sequential phases spanning the academic year. Figure 5.1 depicts the major phases and technical task breakdown.



WORK PLAN – PHASE I

Month	Major Work / Activity	Key Output
JULY 2026	Topic selection, problem identification & initial requirements	Metro surveillance problem, need and project direction defined
AUGUST 2026	Literature survey, research gap & technology study	Relevant papers reviewed; existing approaches, limitations, gap and technologies identified
SEPTEMBER 2026	Literature refinement, objectives, component selection, methodology, architecture, feasibility & project planning	Final literature set, objectives, selected technologies/components, proposed workflow, feasibility, WBS and task allocation finalized
OCTOBER 2026	Post-review technical preparation & Phase-I consolidation	Initial dataset/development preparation and documentation toward the ~40% Phase-I milestone

Figure 5.1: MetroGuard AI Project Work Plan and Major Phase Sequences

The scope of each phase is outlined below:

• **Phase 1 (Literature Study and Requirements):** Comprehensive review of state-of-the-art surveillance models, problem definition, and system requirements analysis.

• **Phase 2 (Dataset Preparation):** Curation and annotation of transit video datasets covering normal flows, physical altercations, crowd surges, and unattended objects.

• **Phase 3 (Detection and Tracking Modules):** Implementation and tuning of YOLOv11s object detection, ByteTrack multi-object tracking, and MobileSAM target tracking (Completed in Phase-I).

• **Phase 4 (Incident Reasoning and Risk Assessment):** Engineering the spatiotemporal correlation matrix, event graph schema, and multi-factor risk priority scoring formula (Phase-II focus).

• **Phase 5 (Dashboard and System Integration):** Developing the full multi-camera dispatcher dashboard with natural-language incident summaries and alert routing.

• **Phase 6 (Testing, Evaluation and Documentation):** End-to-end stress testing on multi-hour transit feeds, benchmark evaluations, final thesis preparation, and defense.

5.2 Gantt Chart and Milestone Timeline

Figure 5.2 illustrates the monthly milestone timeline and Gantt chart across the project duration from September 2026 to April 2027.

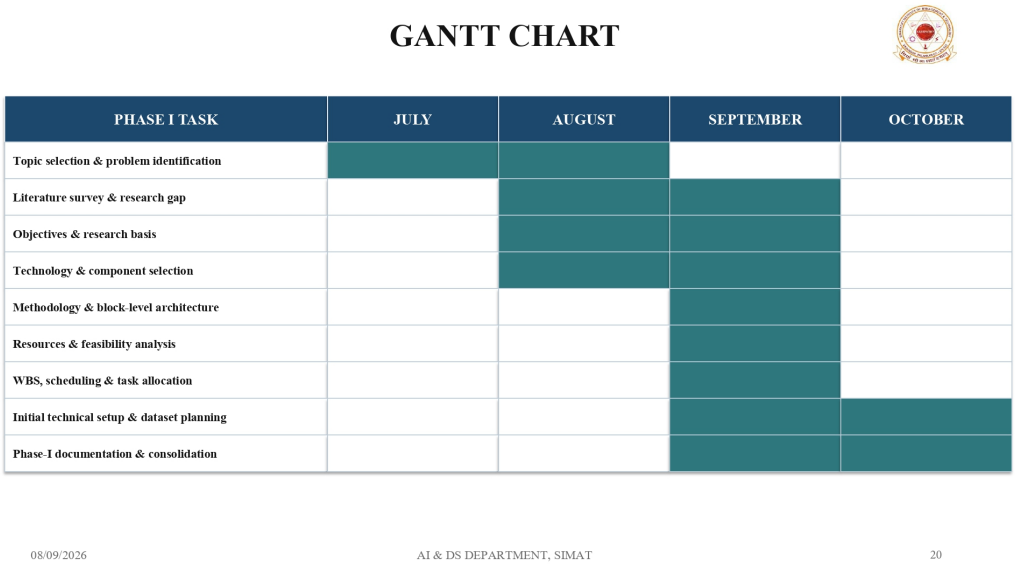


Figure 5.2: MetroGuard AI Project Timeline and Milestone Gantt Chart

As shown in Figure 5.2, Phase-I milestones (Literature Study, Requirements, Detection Backbone, Tracking, and Initial Crowd Dashboard) were successfully accomplished on schedule between September and October 2026. Phase-II execution commences in November 2026.

5.3 Resource Identification

Successful execution of the project relies on the following identified resources:

• **Compute Hardware:** Dedicated workstation equipped with an NVIDIA RTX 4050 GPU (6 GB VRAM) for local edge inference, supported by high-performance multi-core CPUs for video decoding.

• **Video Data Repositories:** Public surveillance benchmark datasets (e.g., UCF-Crime, RWF-2000, PETS 2009, ShanghaiTech Campus) and recorded 1080p transit concourse footage.

• **Software Frameworks:** Python 3.12, PyTorch 2.9, Ultralytics YOLOv11s, ByteTrack C++/Python bindings, MobileSAM, OpenCV, NumPy, and Streamlit.

5.4 Task Allocation Among Team Members

Table 5.1 details the formal allocation of responsibilities among the batch members.

Table 5.1: Task Allocation Among Project Batch Members

Member Name	KTU Reg. No.	Primary Assigned Responsibilities
Adithya Chandran	SPT23AD006	Overall system architecture design, YOLOv11s perception integration, Streamlit operator dashboard engineering, and edge optimization.
Akhil P	SPT23AD009	Surveillance dataset preparation and curation, abandoned baggage module development, model fine-tuning, and empirical accuracy benchmarking.
Muhammed Shibil KP	SPT23AD034	Comprehensive literature review, ByteTrack multi-object tracking integration, and human pose/motion-based fight detection module development.
Shehruz Muhammed CS	SPT23AD043	Spatiotemporal incident reasoning engine, multi-source context fusion, operational risk assessment formulation, and report documentation.
All Members	Joint Team	System integration testing, end-to-end concourse evaluation, technical documentation, and final defense presentation preparation.

5.5 Phase-II Development Roadmap

Building upon the verified Phase-I perception foundation, the Phase-II roadmap encompasses four major technical frontiers:

1. **Fight Detection Integration:** Implement a lightweight pose and optical-flow fusion model (inspired by [8, 14]) to detect physical violence and aggressive grapples in real time.

2. **Abandoned Baggage Tracking:** Deploy the four-state *carry-place-leave-stationary* finite state machine [21] over tracked luggage items, measuring owner-bag separation distance and dwell time.

3. **Graph-Driven Incident Intelligence Engine:** Construct spatiotemporal interaction graphs [11] that dynamically correlate simultaneous altercations, baggage drops, and crowd panics into unified incident entities.

4. **Explainable Natural Language Reporting:** Integrate lightweight language models [5] to synthesize concise, natural-language incident audit reports for rapid transit authority review.

Chapter 6

Conclusion

This final chapter synthesizes the accomplishments of Project Phase-I, highlights the core technical takeaways, and provides concluding perspectives on the future evolution of MetroGuard AI.

6.1 Summary of Phase-I Work

The primary objective of Project Phase-I was to design, construct, and validate a high-throughput, edge-capable computer vision backbone capable of serving as the foundation for the MetroGuard AI multi-event incident intelligence platform. All planned Phase-I objectives were successfully accomplished:

• **High-Speed Commuter Perception:** An Ultralytics YOLOv11s detection backbone was deployed, achieving 79.1% mAP@0.50 on the COCO Person benchmark and maintaining an inference throughput of 62.9 FPS at 1280px resolution on an NVIDIA RTX 4050 GPU.

• **Robust Multi-Object Tracking:** The ByteTrack algorithm was integrated, utilizing dual-score Kalman filtering to maintain persistent commuter identities through severe visual occlusions and crowd intersections.

• **Calibrated Footpoint ROI Filtering:** A normalized Region of Interest framework with ground-contact footpoint filtering was formulated, eliminating perspective distortions and ensuring that only passengers standing within designated operational zones are counted.

• **High-Fidelity Crowd Occupancy Monitoring:** Over an empirical benchmark of 20 sampled surveillance frames encompassing 407 ground-truth pedestrians, the system achieved a remarkable crowd counting accuracy of 99.51% with an MAE of 0.100 pedestrians per frame.

• **Target Tracking Module (TAM):** MobileSAM was integrated to provide prompt-guided silhouette segmentation, dwell time analysis, and kinematic breadcrumb tracking for designated individuals of interest.

• **Interactive Operator Interface:** A responsive Streamlit dashboard was built, providing security personnel with live video streaming, dynamic ROI sliders, occupancy alerts, and an exportable event log.

6.2 Key Takeaways and Contributions

The Phase-I research yielded several key technical takeaways:

• **Edge Viability of YOLOv11s:** Achieving 62.9 FPS confirms that advanced deep learning surveillance does not require expensive data center infrastructure; modern lightweight models can run comfortably on local station workstations.

• **Footpoint vs Centroid Filtering:** In transit concourses with steep camera angles, evaluating the ground-contact footpoint rather than the bounding box centroid is critical to eliminating false boundary crossings.

• **Foundation for Multi-Event Fusion:** By establishing persistent tracking and robust occupancy estimation, the system possesses the exact spatial-temporal primitives required to link fights, crowds, and abandoned baggage in Phase-II.

6.3 Concluding Remarks

The successful completion of Project Phase-I establishes a validated, industrial-grade perception pipeline for smart metro surveillance. MetroGuard AI demonstrates that edge computer vision can deliver sub-second processing speeds, exceptional counting fidelity, and real-time situational awareness. With this robust foundation in place, Project Phase-II will focus on integrating physical altercation detection, unattended baggage tracking, graph-based context fusion, and explainable incident generation—ultimately delivering a transformative, multi-event decision-support platform for modern urban transit systems.

References

[1] A. A. Hamid, S. A. Monadjemi, and B. Shoushtarian, 'ABDviaMSIFAT: Abnormal Crowd Behavior Detection Utilizing a Multi-Source Information Fusion Technique,' *IEEE Access*, vol. 13, pp. 75000–75019, 2025, doi: 10.1109/ACCESS.2024.3436007.

[2] D. Chaudhary, S. Kumar, and V. S. Dhaka, 'Intelligent Anomaly Detection and Localization With Crowd Density Estimation for Improved Public Safety,' *IEEE Access*, vol. 13, pp. 169912–169928, 2025, doi: 10.1109/ACCESS.2025.3614146.

[3] A. M. Qasim, N. Abbas, A. Ali, and B. A. A. Al-Ghamdi, 'Abandoned Object Detection and Classification Using Deep Embedded Vision,' *IEEE Access*, vol. 12, pp. 35539–35551, 2024, doi: 10.1109/ACCESS.2024.3369233.

[4] H.-Y. Lin and C.-H. Tseng, 'Abnormal Activity Detection and Classification of Bus Passengers With In-Vehicle Image Sensing,' *IEEE Access*, 2024, doi: 10.1109/ACCESS.2024.3365138.

[5] T. Yuan, X. Zhang, K. Liu, B. Liu, C. Chen, J. Jin, and Z. Jiao, 'Towards Surveillance Video-and-Language Understanding: New Dataset, Baselines, and Challenges,' in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 22052–22061, doi: 10.1109/CVPR52733.2024.02082.

[6] S. F. A. Shah, M. Q. Khan, A. A. Alsuwaylimi, and L. A. Alenezi, 'A Two-Stage Spatiotemporal CNN-YOLOv9 Framework for High-Precision Real-Time Abandoned Object Detection in Public Surveillance Videos,' *IEEE Access*, vol. 14, pp. 35983–35997, 2026, doi: 10.1109/ACCESS.2026.3669588.

[7] Y. Wang and V. Y. Mariano, 'A Multi Object Tracking Framework Based on YOLOv8s and ByteTrack Algorithm,' *IEEE Access*, 2024, doi: 10.1109/ACCESS.2024.3450370.

[8] S. T. Y. Aung, W. Kusakunniran, and Y. K. Hooi, 'Leveraging Fusion Methods of Human Pose and Motion Dynamics for Accurate Violence Detection in Video Surveillance,' *IEEE Access*, 2025, doi: 10.1109/ACCESS.2025.3613765.

[9] Z. Yang and R. J. Radke, 'Context-aware Video Anomaly Detection in Long-Term Datasets,' in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2024, pp. 4002–4011, doi: 10.1109/CVPRW63382.2024.00404.

[10] Z. Zhu, H. He, and M. Wang, 'A Spatio-Temporal Semantic Algorithm for Video Anomaly Detection from Monitoring Perspectives,' in *Proc. 5th Int. Conf. Artif. Intell., Robot., Commun. (ICAIRC)*, 2025, doi: 10.1109/ICAIRC68035.2025.11385035.

[11] M. Khan, W. Gueaieb, A. Elsaddik, G. De Masi, and F. Karray, 'Graph-Based Knowledge Driven Approach for Violence Detection,' *IEEE Consum. Electron. Mag.*, vol. 14, no. 1, pp. 77–85, 2025, doi: 10.1109/MCE.2024.3446192.

[12] M. Khan, M. Saad, A. Khan, W. Gueaieb, A. E. Saddik, G. De Masi, and F. Karray, 'Action Knowledge Graph for Violence Detection Using Audiovisual Features,' in *Proc. IEEE Int. Conf. Consum. Electron. (ICCE)*, 2024, doi: 10.1109/ICCE59016.2024.10444158.

[13] A. S. Kannan, Z. A. Alsalami, A. Nihlani, K. Gowthami, M. A. K. Abdukodir Kizi, A. Amudha, and Z. Mamadalieva, 'Real-Time Video Surveillance Analysis at the Edge in Smart Cities,' in *Proc. Int. Conf. Innov. Emerg. Technol. AI Commun. Syst. (IETACS)*, 2025, doi: 10.1109/IETACS68750.2025.11385417.

[14] M. Khan, A. E. Saddik, W. Gueaieb, G. De Masi, and F. Karray, 'VD-Net: An Edge Vision-Based Surveillance System for Violence Detection,' *IEEE Access*, vol. 12, pp. 43796–43808, 2024, doi: 10.1109/ACCESS.2024.3380192.

[15] M. H. Habeb, M. Salama, and L. A. Elrefaei, 'An Attention-Based Unsupervised Framework for Video Anomaly Detection in Large Heterogeneous Environments,' in *Proc. MIUCC*, 2024, doi: 10.1109/MIUCC62295.2024.10783549.

[16] H. Lee, D. Kang, H. Park, S. Park, D. Jeong, and J. Paik, 'Real-Time Human Group Detection and Clustering in Crowded Environments Using Enhanced Multi-Object Tracking,' *IEEE Access*, vol. 12, pp. 184028–184039, 2024, doi: 10.1109/ACCESS.2024.3503661.

[17] M. Zhang, J. Wang, Q. Qi, H. Sun, Z. Zhuang, P. Ren, R. Ma, and J. Liao, 'Multi-Scale Video Anomaly Detection by Multi-Grained Spatio-Temporal Representation Learning,' in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 17385–17394, doi: 10.1109/CVPR52733.2024.01646.

[18] A. Amirkhani and S. A. Khoshnevis, 'CFTformer: End-to-End Cross-Frame Multi-Object Tracking With Transformer,' *IEEE Access*, 2025, doi: 10.1109/ACCESS.2025.3567349.

[19] S. Zhuo, H. Bai, L. Jiang, X. Zhou, X. Duan, Y. Ma, and Z. Zhou, 'SCL-YOLOv11: A Lightweight Object Detection Network for Low-Illumination Environments,' *IEEE Access*, vol. 13, pp. 47653–47662, 2025, doi: 10.1109/ACCESS.2025.3550947.

[20] H. Zhang, Y. Guo, and X. Du, 'Online Action Detection by Long Short-Term Transformer with Query Exemplars,' in *Proc. IEEE Int. Joint Conf. Neural Netw. (IJCNN)*, 2024, doi: 10.1109/IJCNN60899.2024.10651333.

[21] T. Soontornnapar, 'Abandoned Bag Detection in Public Areas Using Grounding DINO With Fine-Grained Prompts,' in *Proc. ECTI-CON*, 2025, doi: 10.1109/ECTI-CON64996.2025.11100886.